Voor jouw apotheek

Svelte Fundamentals

Module – Using HTTP

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

Communicating with API’s

Using HTTP in your Svelte apps

Consuming data from an API

- We're going to talk to a 'real' country-API on the internet
  - https://restcountries.com/
- Using `http` is *not* built into Svelte

Two popular options

- Using native `fetch()`
  - Built-in in the browser
  - No extra dependencies
  - Aligns with usage in SvelteKit
- Using Axios
  - More options than native `fetch()`
  - Popular & trusted library
  - No conversion to `json` necessary
  - Used by Fortune 500-companies

Using `fetch()`

- Pure Svelte app:
  - Use in `onMount()` lifecycle hook → data appears when the component is loaded
  - Store `loading` / `error` / `success` state in local component state
  - Render component based on state.
- SvelteKit app (later on):
  - Use `load()` in the page. SvelteKit uses own `fetch()` implementation from there.
  - Options in SvelteKit: use form actions/server endpoints for Server Sided Fetching (pure Svelte: client sided fetching)

Optional: using axios

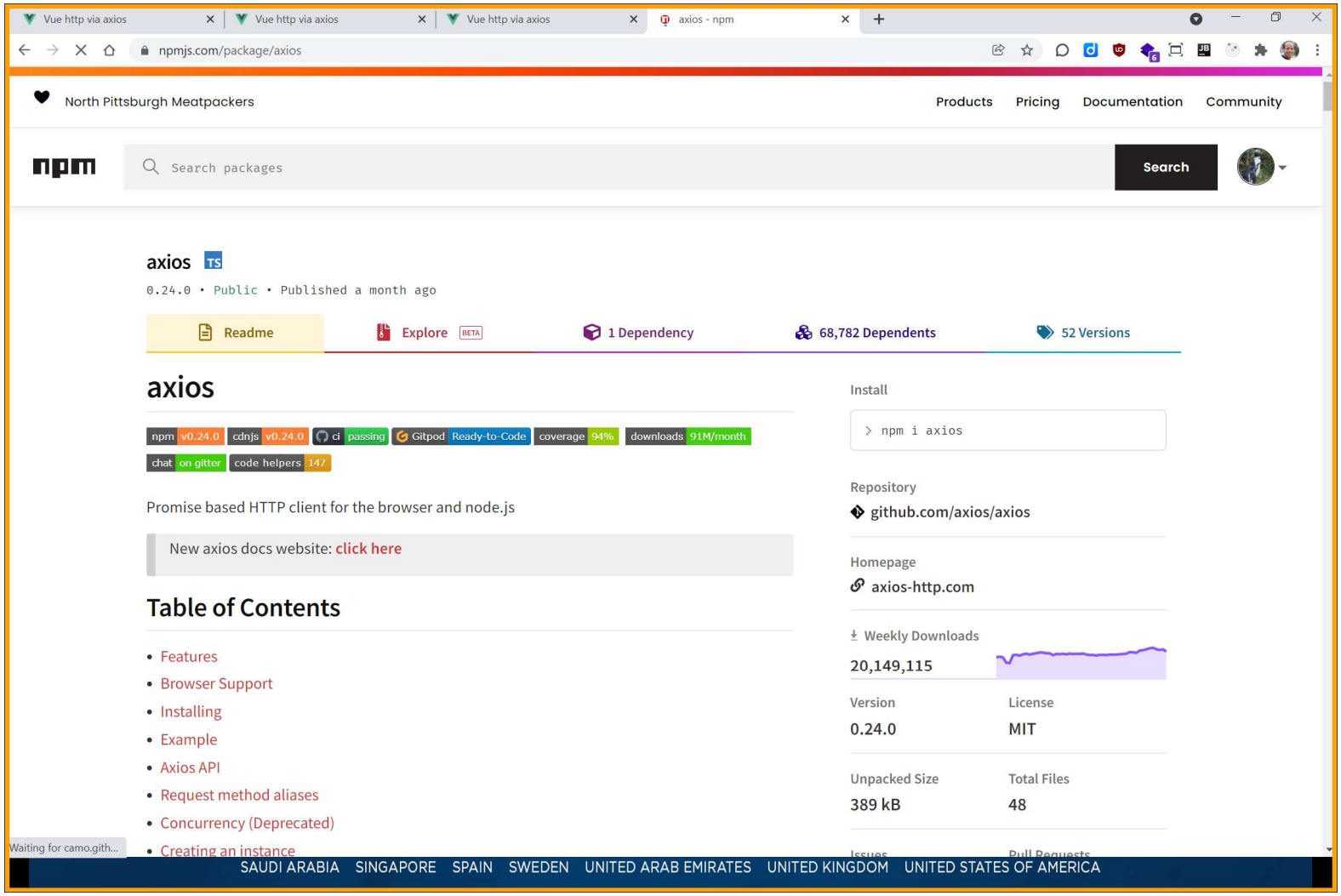
When you don’t want to use `fetch()`

Using Axios

"There are many times when building application for the web that you may want to consume and display data from an API. There are several ways to do so, but a very popular approach is to use ***axios****, a promise-based HTTP client."*

https://github.com/axios/axios

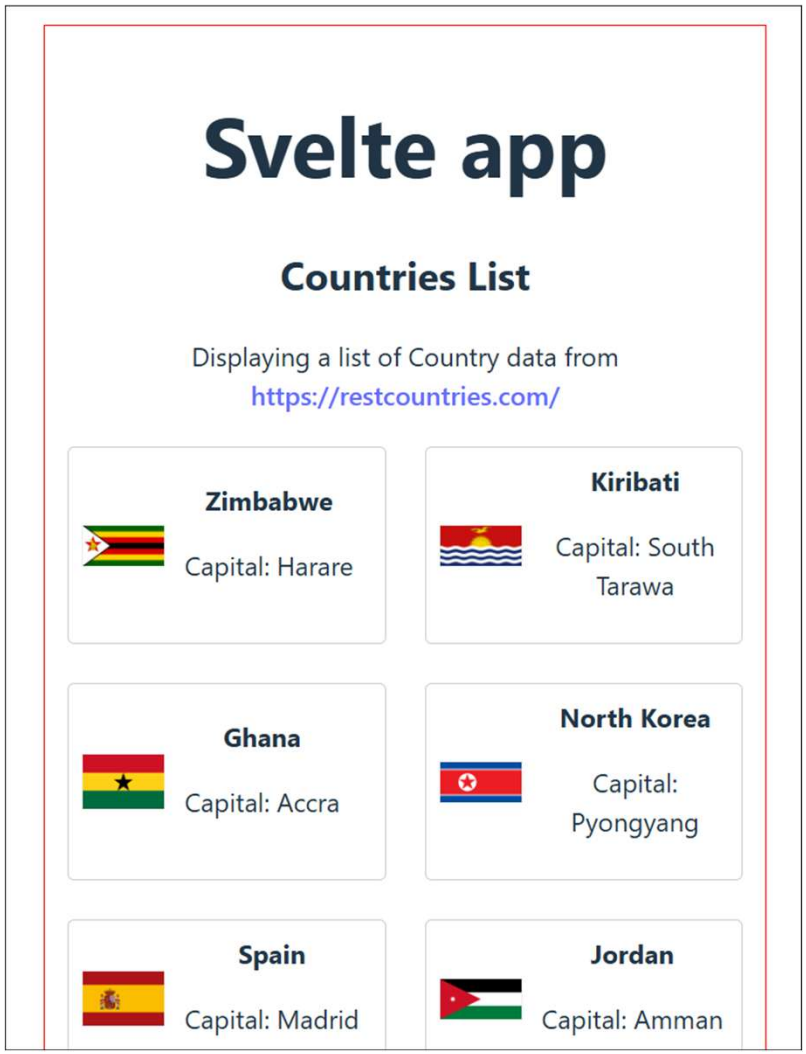
https://www.npmjs.com/package/axios



Creating your component(s)

Talking to an http-backend, storing results in local state

Final result – what are we creating?



When the app/component is loaded, data is fetched on startup and displayed in a list

Creating variables for state

- We need to account for states:
  - `loading` – busy loading data (show spinner/indicator)
  - `done` – fill arry with fetched data
  - `error` – show error in case of failure
- Data model – we call it `Country.ts`, something like

```
export interface Country {
    name: {
        common: string;
        …;
    };
    capital: string[];
    flags: {… };
}
```

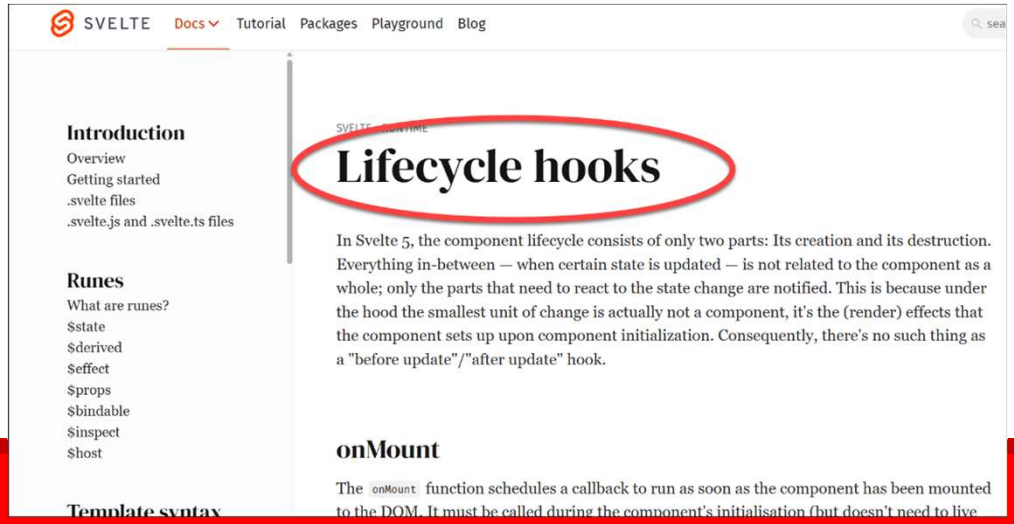
Local State

- For now: local state variables.
  - Future: store state in a `Store` or global `$state()`

```
// local state, prepared for reactive updates
let countries: Country[] = $state([]);
let loading: boolean = $state(true);
let error: string | null = $state(null);
```

Lifecycle Hooks

- Svelte has some default lifecycle hooks available
  - `onMount()` – when the component is created
  - `onDestroy()` – when component is destroyed
  - `tick()` – everything in between (DOM-updates for instance).
- These are just functions that are called automatically
- https://svelte.dev/docs/svelte/lifecycle-hooks



Calling an API on startup

- Import the correct lifecycle hook and write code

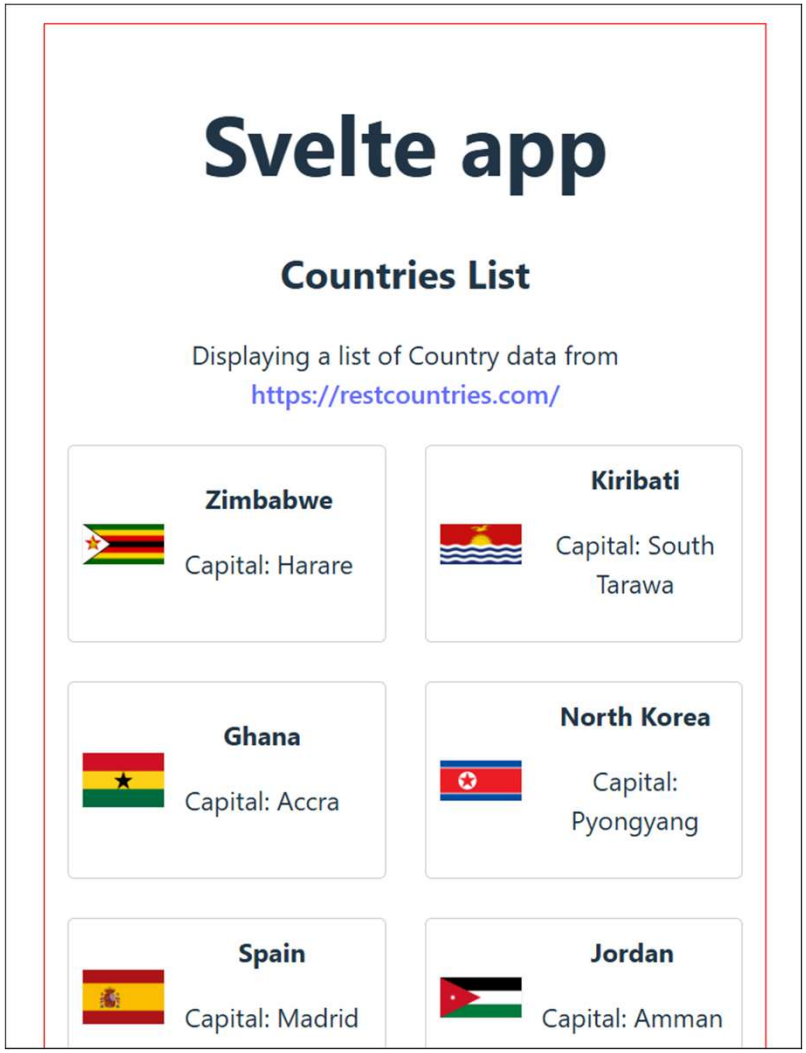
```
onMount(async () => {
    try {
        // Simulated delay of 1500ms to show loading state
        await new Promise(resolve => setTimeout(resolve, 1500));
        const response = await fetch('https://restcountries.com/...');
        if (!response.ok) {
            throw new Error(`Failed to fetch: ${response.statusText}`);
        }
        countries = await response.json();
    } catch (e) {
        error = e instanceof Error ? e.message : 'An unknown error occurred';
    } finally {
        loading = false;
    }
});
```

Create UI

- Create UI and account for all possible states.
- Write HTML and CSS to make it look nice.

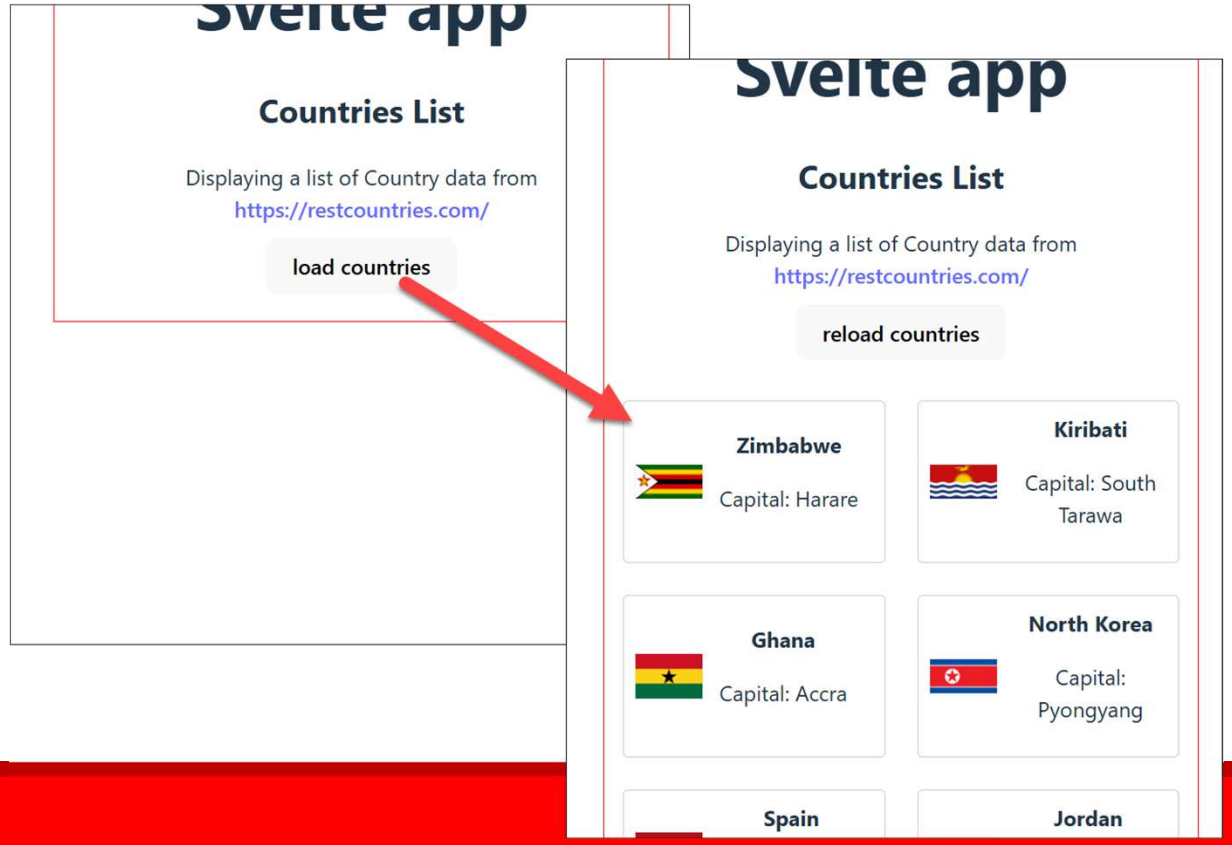
```
{#if loading}
    <div class="loading-container">
        …
    </div>
{:else if error}
    <p style="color: red;">Error: {error}</p>
{:else}
    {#each countries as country}
        …
    {/each}
{/if}
```

Result


Svelte app
Countries List
Displaying a list of Country data from
https://restcountries.com/
Zimbabwe
Capital: Harare
Kiribati
Capital: South Tarawa
Ghana
Capital: Accra
North Korea
Capital: Pyongyang
Spain
Capital: Madrid
Jordan
Capital: Amman

Delegating to `click` event

- If we only want to talk to backend in case of a specific event
  - For instance: mouseclick on a button
- Use `onclick` and create (`async`) function as usual



Function and HTML

```
async function loadCountries() {
    loading = true;
    error = null;
    try {
        const response = await fetch(url);
        …
    } catch (e) {
        error = e instanceof Error ? e.message : 'An unknown error occurred';
    } finally {
        loading = false;
    }
}
```

```
<button onclick={loadCountries} disabled={loading}>
    {hasLoaded ? 'reload countries' : 'load countries'}
</button>
```

../examples/200-http

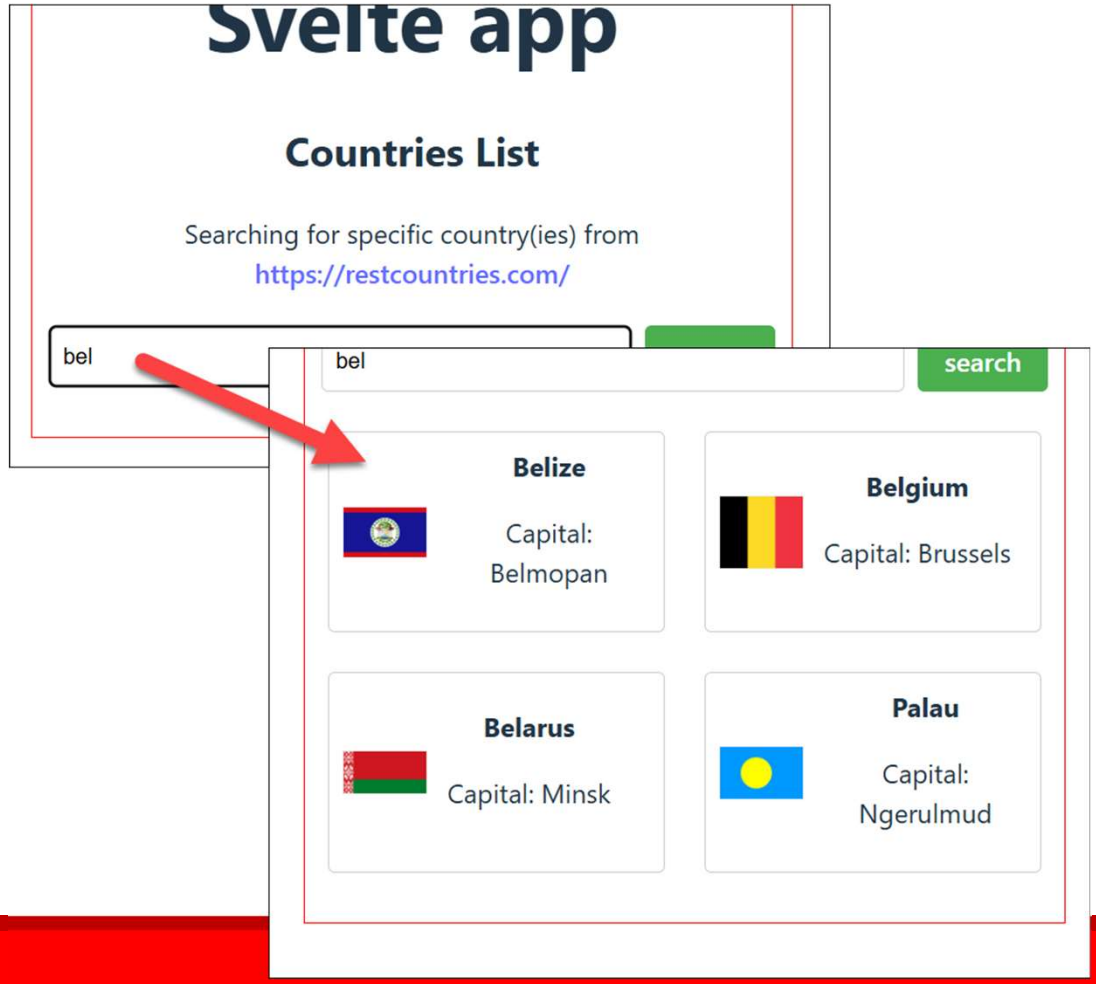
Searching an API

Using a textbox to send parameters to a backend –

(of course there must be an endpoint available!)

Goal

- Create searchbox on the page, only search for specific data in the API



Binding to a state variable

```
// string holding the search term from the input box
let searchTerm: string = $state('');
```

```
// Handler for the Enter key press
const handleKeyDown = (event: KeyboardEvent) => {
    if (event.key === 'Enter') {
        searchCountries();
    }
}
```

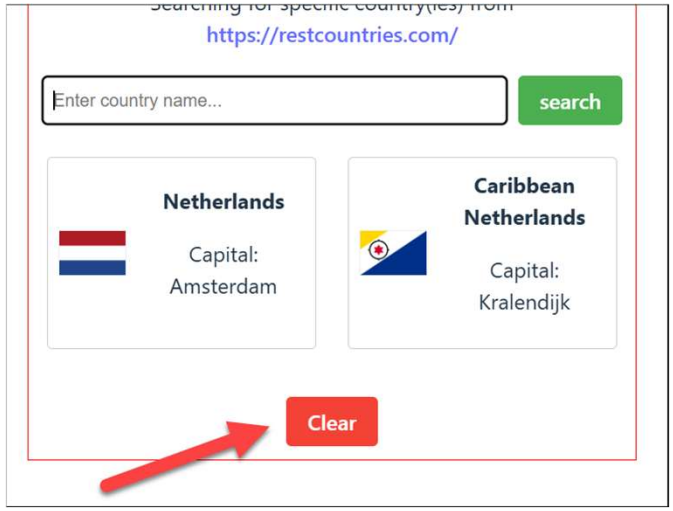
```
<input
        type="text"
        bind:value={searchTerm}
        onkeydown={handleKeyDown}
        placeholder="Enter country name..."
/>
```

Also: update the endpoint, adjust `search()`

- Actually, only endpoint differs. The rest of the UI is basically the same
  - Of course, this depends on the API you are using!

```
const url = `https://restcountries.com/v3.1/name/{name}?...`;
```

../210-http-search

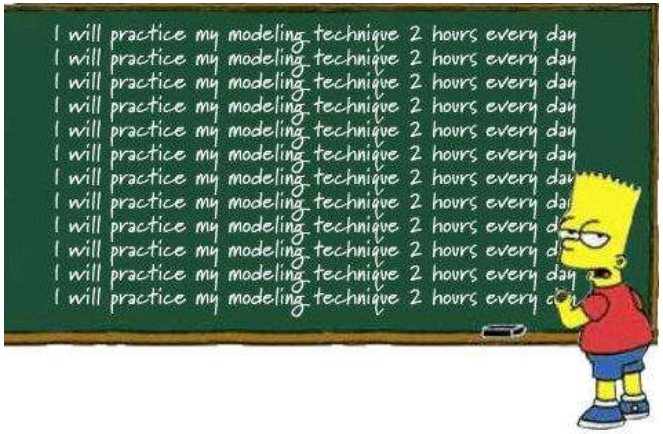


Example API's

- https://pokeapi.co/ - Pokemon API
- https://jsonplaceholder.typicode.com/ random users, posts, photos
- http://www.omdbapi.com/ - Open Movie Database
- http://swapi.dev/ - Star Wars API
- https://cataas.com/ - Cat as a Service
- See also `JavaScript APIs.txt` with other API's.

Workshop

- Pick one of your own projects, or see for instance:
  - `../210-http-search`
- Create a small application using one of the API's in the file `JavaScript API's.txt`, using `fetch()`, for example
  - Pokemon API
  - Kenteken API
  - OpenWeatherMap API
  - …



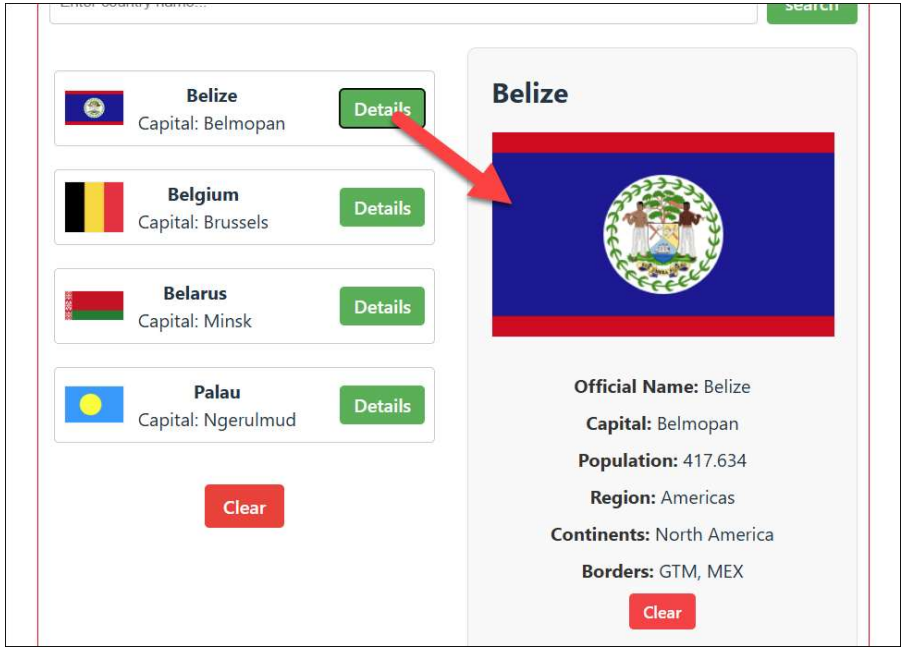
Fetching details

Doing additional lookups after specific events, e.g. `click`

(again, there must be an endpoint available!)

Clicking a specific country – fetching details

- Handle the `onclick` event for a country
- Do additional lookup in the Smart component
- Pass details to View component `CountryDetail`



Concept:

Design Pattern

Smart/View Components

“In Svelte, all components are equal. There is no sense of different ‘types’ of components”

Smart components / View Components

- Design pattern
- Why ? Separation of concerns
  - View component is responsible for presentation (and *can* be used in a completely different environment with different component logic)
  - Smart component is responsible for logic → passing the [calculated] data to the view component.
- Smart components
  - AKA: *Container* components, *controller* components, *statefull* components
- View components
  - AKA: *pure* components, *dumb* components, stateless

Characteristics

- Smart components
  - Typically contain big(ger) chunks of logic
  - Typically have a small UI part (reference just the view component)
  - Pass the data to the view component
- View components
  - Typically contain no logic whatshowever
  - Get their data via `@Input()` decorators
  - Submit events via `@Output()` decorators
  - Have large chunks of UI

Smart Component – add logic

- Add logic to fetch details from endpoint and clear current selection

```
// 0a. More local state, handling loading of country details.
let selectedCountry: Country | null = $state(null);
let loadingDetail: boolean = $state(false);
…
const detailUrl = `https://restcountries/…/{name}?fullText=true&…`;
```

```
<!-- In the main list we add a button to search for details-->
<button
        class="details-button"
        onclick={() => fetchCountryDetail(country.name.common)}
        type="button">
    Details
</button>
```

Smart component – pass data down

- Pass fetched data to the View component, using bindings in the template.
- We also pass a function to be executed by the View Component
- The rest (`{#if selectedCountry}` etc) is just there to create a better UX
  - No functional or logical meaning

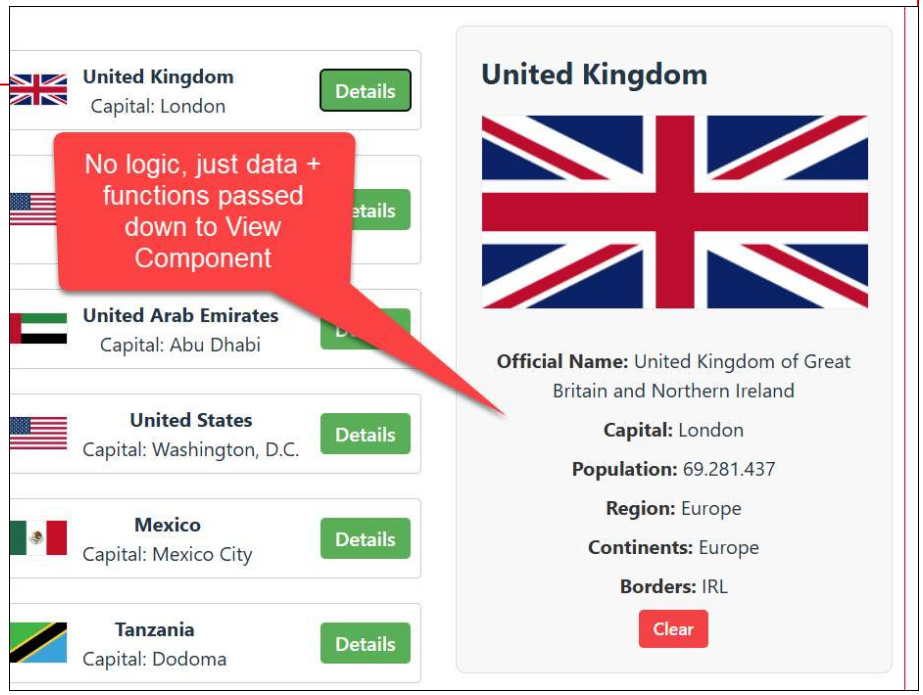
```
<CountryDetail
    country={selectedCountry}
    onClear={() => selectedCountry = null} />
```

View Component – only show data

```
<!--CountryDetail.svelte - displaying details of a specific country-->
<script lang="ts">
    import type {Country} from './types/Country';

    // Proper types for the props passed to this component.
    interface Props {
        country: Country;
        onClear: () => void;
    }

    // destructure the props passed to this component
    let {country, onClear}: Props = $props();


    // no further logic in this component
</script>
```

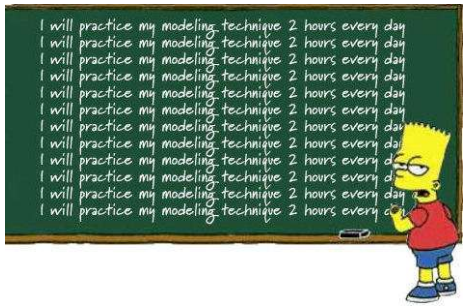
Template of the View Component

```
<!--UI/HTML for this component, taking the state into account. No logic.-->
<div class="country-detail">
    <div class="header">
        <h2>{country.name.common}</h2>
    </div>
    …
    <button class="detail-clear-button" onclick={onClear}>Clear</button>
</div>
```



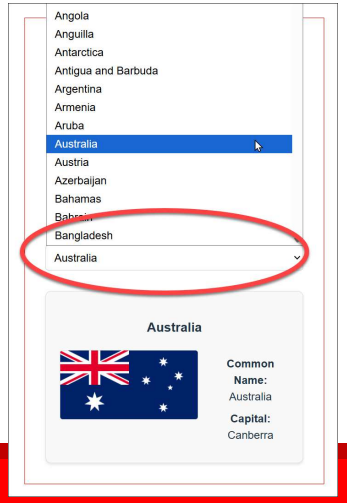
Workshop

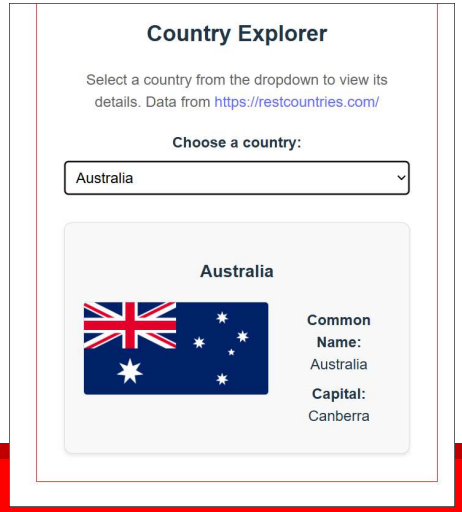
- Work from: `../220-http-details`, or use your own project
- Create a new Smart component from the current `CountryList.svelte`. Pass data to additional View Components:
  - The Search box
  - The Country List
  - Country Detail
- So end up we end up with 1 Smart- and 3 View Components
- What will the architecture look like?

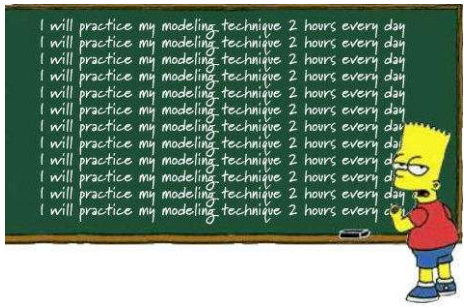


Combination Workshop

- Work from: `../200-http`, or use your own project
- On startup, call the API and load a dropdown list with available countries, sorted alphabetically
- This basically combines all knowledge:
  - 1. Load application ➔ 2. talk to API ➔ 3. store results in state ➔ 4. show state in a dropdown in the UI ➔ 5. update UI when state changes.







Checkpoint

- You know about using http-endpoints in Svelte
- You can make calls and show response in the UI
- You can make additional calls and retrieve detailed data
- You are aware of the Smart/View Component design pattern and can apply this to Svelte applications.